

From Proof to Production: Operating the Provability Fabric

SentinelOps

September 1, 2025

Abstract

This whitepaper is a *runbook for operators*. It explains how to deploy, measure, and troubleshoot the **Provability Fabric (PF)** in production: the sidecar-mediated runtime that enforces proof-backed policies for agentic systems. Unlike our *Ecosystem/Standards* paper (verification-by-default) and our *Mechanics* paper (PF semantics, compilation, and proofs), this document focuses on **operational practice**: deployment topologies, **obs→shadow→enforce** rollouts, SLOs and dashboards, replay determinism, incident response, reject budgets, break-glass governance, concurrency/clock assumptions, and performance envelopes. It includes concrete artifacts (evidence bundle layout, replay kit schema, CI gates, Grafana panels) and codifies the *monitor-to-claim* bridge as an operational contract with preconditions SREs can verify.

Contents

1	Executive Summary for Operators	3
2	Deployment Patterns	3
2.1	Topologies	3
2.2	Frozen effects and adapters	4
3	Operating Model: observe→shadow→enforce	5
3.1	Observe (obs)	5
3.2	Shadow (shadow)	6
3.3	Enforce (enforce)	6
4	Operational Contract: Monitor→Claim	6
4.1	What you get if you meet the preconditions	6
4.2	Concurrency clocks (what Ops must pin)	7

5	Deterministic Egress Replay Determinism	8
5.1	Profile	8
5.2	Replay kit	8
5.3	Residual channels	8
6	Evidence, SLOs, and Dashboards	9
6.1	Evidence bundle (per release)	9
6.2	Per-emission certificate (runtime)	9
6.3	SLOs	10
6.4	Dashboards	10
7	Runbooks	11
7.1	CI gates	11
7.2	Rollout: clause by clause	11
7.3	Break-glass	12
7.4	Incident response	12
8	Minimal Taint Discipline for RAG	13
9	Cost Performance	13
10	Security, Privacy, and Governance Controls	14
11	Limitations and Risks	14
12	Roadmap	15
13	Conclusion	15

1 Executive Summary for Operators

Provability Fabric (PF) provides effect-level enforcement—*deny-wins*—across calls/files/net/de-class, issues verifiable certificates for every emission, enforces deterministic egress to enable faithful replays, unifies permissions, and supplies a defensible bridge from online monitor acceptance to global confidentiality claims under declared assumptions.

Adoption follows a three-phase path. Teams begin in **observe** mode, where nothing is blocked and traces are exported to reveal the true effect surface. They then proceed to **shadow** mode to compare proposed monitor decisions against live outcomes in situ. Finally, they selectively **enforce** clauses behind an explicit *reject budget* so that assurance increases without jeopardizing availability.

Measurement is continuous and tied to explicit SLOs: certificate-verification rate, replay determinism, p95 monitor latency, reject-budget adherence, and inclusion-proof latency. Standard dashboards and CI gates are provided to surface these signals, prevent regressions, and make conformance auditable.

Recovery procedures are deliberate and repeatable. Replays are byte-faithful under declared profiles; counterexample shrinkers produce minimal traces for diagnosis; break-glass is M-of-N with short time-to-live and automatic paging; and every incident ships with a replay kit so independent parties can re-derive the monitor’s verdicts.

2 Deployment Patterns

2.1 Topologies

Selecting a deployment topology determines how strictly effects are mediated, how faults are contained, and how easily the system can satisfy replay and evidence requirements. In all cases the SC is the locus of enforcement and evidence production; the difference lies in where it runs and what it is responsible for mediating.

Sidecar-per-agent (default). In the default topology, each agent process is paired with a dedicated SC that intercepts file system, network, process, and tool invocations through language adapters and proxy hooks. This arrangement maximizes isolation: a failure or denial affecting one agent does not perturb others, and certificates, logs, and replays remain cleanly attributable to a single identity and configuration. Because every effect originates from exactly one agent–sidecar pair, incident diagnosis is straightforward and counterexamples are minimal. The operational costs are modest but real: there is some per-agent memory and CPU overhead, and fleet management must ensure that the SC and agent are version-pinned

together. This topology is preferred for multi-tenant clusters, regulated workloads, and any environment where fault containment and audit attribution are more important than absolute minimum resource usage.

Host sidecar (edge). On constrained devices or single-tenant edge nodes, a privileged SC can mediate multiple short-lived agent processes on the same host. Agents are sandboxed with namespaces and cgroups; the SC enforces admission controls, per-agent identities, and per-agent rate/byte budgets while sharing code and caches across tenants on the node. This reduces footprint and startup time, which is valuable for bursty workloads and embedded hardware. The trade-off is a larger blast radius if the host SC is misconfigured, so operators must harden the host boundary (e.g., seccomp and mount profiles), tag all evidence with stable per-agent identifiers, and monitor isolation invariants continuously. This topology is appropriate when resources are tight and the node’s trust boundary is already well controlled.

Service mesh (cloud). In mesh-enabled clusters, the SC integrates with the service mesh to mediate ingress and egress at the network layer while local shims continue to guard file system and process effects. The mesh supplies mTLS, identity, and traffic policy; the SC contributes certificate generation, declassification checks, quotas, and deterministic egress shaping. This division of labor lowers duplication and allows platform teams to reuse existing observability and policy planes. To maintain the proof context, operators must ensure that all network paths traverse mesh hooks (no host-network bypass), declare the egress profile at deployment time, and keep local adapters in place for non-network effects. This topology fits cloud services that already standardize on a mesh and seek consistent mediation without rewriting platform primitives.

2.2 Frozen effects and adapters

Regardless of topology, all observable effects must pass through *frozen* adapters whose behavior is explicitly specified and versioned. Freezing adapters ensures that what is proved about mediation is exactly what runs in production and that replays remain faithful to the original execution.

HTTP GET/POST. Network adapters pin DNS resolution to approved resolvers and restrict destinations to an allow-listed set of URLs or domains. They enforce **Content-Length** and forbid implicit redirects unless explicitly declared in policy, and they require explicit codecs for request and response bodies. These constraints remove ambiguity that could

undermine reproducibility, reduce covert timing and length channels, and make certificate fields (arguments, lengths, and verdicts) meaningful during audit and replay.

File read/write. File system adapters expose only declared paths via read-only or read-write bind mounts, normalize paths before access, and reject symlinks and other indirections that could evade policy. Each access records inode, device, and content hash so that replays can verify the exact bytes read or written. These practices prevent path-traversal tricks, preserve finiteness at the monitor, and provide strong anchors for evidence that a given emission was justified by the declared policy and state.

Tool RPC. Tool invocations are mediated by gRPC or HTTP interceptors that validate schemas at the boundary, attach field-level labels, and verify per-field witnesses (for example, Merkle proofs of membership for selected paths and bounded taint derivations for joins). The adapter emits a single, typed event on the alphabet per call, accompanied by the witnesses required for the monitor to decide. Versioned adapter contracts ensure that changes in tool semantics are visible to policy and proof code, keeping the runtime behavior aligned with the obligations that auditors later verify.

3 Operating Model: **observe**→**shadow**→**enforce**

3.1 Observe (**obs**)

The **observe** phase exists to reveal the true effect surface of an agent and to ground policy writing in facts rather than assumptions. Operators enable the SC with `-mode=observe` so that every candidate effect is intercepted, recorded, and explained, but never denied. In this mode the system exports complete traces together with *certificate drafts* that show which hashes, witnesses, and verdict fields would have been produced had enforcement been active. To make those traces replayable from day one, deterministic egress (§5) should be enabled with conservative chunk size and flush cadence so that timing and length artifacts are controlled during measurement. In parallel, teams light up dashboards that track coverage, proposed denials by clause, and monitor latency (Table 6.4); these dashboards establish initial SLO baselines (Table 6.3) against which subsequent phases will be judged. By the end of **observe**, policy authors know which effects exist, which labels and witnesses are missing, and how frequently each clause would have fired under realistic load.

3.2 Shadow (**shadow**)

The **shadow** phase compares, in situ, what the system *would* do under the current policy to what it *actually* does in production. Running with `-mode=shadow` causes the SC to emit both a *live* verdict (always **admit** at this stage) and a *proposed* verdict computed by the current monitor tables. Differences are logged with precise counterexamples and graphed over time so that teams can see which rules are noisy, where quotas are too tight, and which declassification budgets are either overly permissive or unrealistically strict. Policy changes in this phase focus on calibrating quotas and declassification budgets until the measured false-deny rate falls below the target bound established during **observe**. Because deterministic egress is already in place, every discrepancy can be replayed byte-for-byte, allowing authors to fix specific clauses rather than guessing at aggregate behavior. The exit criterion for **shadow** is a stable gap close to zero between proposed and live decisions on representative workloads.

3.3 Enforce (**enforce**)

Only once the proposed decisions match operational intent do we flip selected clauses to enforcement. The **enforce** phase denies violations online while preserving availability by operating behind an explicit *reject budget*. In practice, teams promote clauses gradually and declare a budget such as ‘no more than 25 denials per hour per service,’ which bounds the blast radius if a policy regresses. At the same time, a tightly governed *M-of-N break-glass* path is wired in with a short time-to-live, a required human-readable justification, and on-call paging so that safety can be temporarily relaxed under supervision without silently eroding guarantees. Continuous integration gates deployments on both conformance and SLOs: builds must pass the replay and certificate suites, and the `proof_hash` and `automata_hash` embedded in artifacts must match those observed at runtime. When these conditions hold, denials become routine, bounded events rather than incidents.

4 Operational Contract: Monitor→Claim

4.1 What you get if you meet the preconditions

The operational promise of the system is a clear bridge from online monitor acceptance to an auditable, offline claim about the traces your agents produce. Informally, if every effect is mediated on the declared alphabet, if field labels and declassification witnesses are correct for each emission, if the monitor tables in production match the hashes bound into the PAB, and if there are no out-of-band channels beyond the declared egress profile, then acceptance of all

prefixes by the online monitor entails that the enforced safety policy holds on the resulting traces.

Theorem 1 (Operational Soundness Bridge). *Let Φ be the enforced safety policy with declassification discipline D . If (i) all effects are completely mediated on the declared alphabet, (ii) field labels and declassification witnesses are correct for each emission, (iii) the monitor tables match the PAB proof and policy hashes, and (iv) there are no out-of-band channels beyond the declared egress profile, then whenever the online monitor accepts all prefixes, the produced traces satisfy Φ (for example, TI-NI at the declared level).*

In practice, the site reliability engineering runbook stands in for these premises: mediation hooks and adapters are verified, labelers are checked by dual implementations, egress parameters are pinned, and hashes are validated during deploy. When the checklist is green and the monitor accepts, audit teams can escalate the acceptance result into a formal claim with confidence.

4.2 Concurrency clocks (what Ops must pin)

Correctness at runtime depends on a documented scheduling and clock model that the platform actually enforces. The default scheduling model is a *single global event queue* (SGEQ) that linearizes all observable effects and feeds them to the monitor in order. In environments where throughput requires more concurrency, a two-queue variant that separates tool calls from egress can be used safely provided that cross-queue events commute with respect to the guard and monitor semantics and that explicit serialization points are introduced at well-defined barriers. Under these conditions, the following preservation result applies.

Lemma 1 (Two-queue preservation). *Given an independence relation I and FIFO-preserving merges at serialization barriers, any merged trace that is accepted by the per-queue monitors is equivalent, up to commuting independent events, to a trace accepted by a single global monitor.*

The clock model must likewise be pinned. Operators should use a monotonic clock with a documented tolerance ε and widen rate-limit windows from Δ to $\Delta + \varepsilon$ so that enforcement is robust to timer jitter and scheduling variance. The API and tolerance belong in the runbook so that replay harnesses and audits use exactly the same assumptions as production.

5 Deterministic Egress Replay Determinism

5.1 Profile

Deterministic egress is a runtime discipline that fixes the observable degrees of freedom in streaming output so that independent parties can reproduce executions faithfully. In practice, operators declare a profile that pins the byte-level chunking policy and the emission cadence by setting `-chunk-bytes` and `-flush-ms`. The profile also fixes text formatting by pinning the locale and the timezone, which prevents drift introduced by number, date, or collation differences. Some deployments additionally enable length padding to bound residual channels; when padding is active, the policy must state whether it pads to a fixed length or to a step-function envelope. Every run produced under a deterministic egress policy records a unique profile identifier, and that identifier appears in certificates, logs, and evidence bundles so that auditors can select the correct parameters for replay without guessing.

5.2 Replay kit

A replay kit turns evidence into an executable artifact that a third party can use to re-derive monitor verdicts and reproduce incidents. Each kit contains (i) fixtures such as input corpora and adapter stubs that replace live dependencies; (ii) environment pins for the declared profile, including locale, timezone, chunk size, flush cadence, random seeds, tokenizer versions, and any adapter configuration required to freeze effects; (iii) canonical traces captured at runtime with indexes into the corresponding certificate records; and (iv) oracles that state, for each trace, whether the monitor must accept or deny and which quotas or declassification budgets apply. With these materials in hand, an auditor can rehydrate the environment, replay the trace byte-for-byte, and obtain the same monitor outcomes as production, with any divergence called out explicitly. A brief checklist describing directory layout and invocation commands is included in the kit to minimize setup friction (see the reproducibility checklist in the appendices for a suggested template).

5.3 Residual channels

Determinism constrains but does not eliminate channels that leak information through timing and length. The fixed chunk size and flush cadence make those channels measurable, which allows operators to publish upper bounds on residual capacity in bits per second together with the measurement method used to obtain them. When padding is enabled, the policy should document the trade-off curve between capacity reduction and false-block rate so that changes to the padding envelope can be justified operationally. These disclosures make the

remaining risk legible to auditors and enable like-for-like comparisons across deployments that claim similar assurances.

6 Evidence, SLOs, and Dashboards

6.1 Evidence bundle (per release)

Each release ships with an evidence bundle that binds runtime behavior to build-time artifacts and makes verification routine. The bundle enumerates the `policy_hash`, `proof_hash`, `automata_hash`, `labeler_hash`, so that the monitor and labeler in production can be matched to the proofs and specifications that justify them. It includes representative certificate samples and, where applicable, transparency-log references so that inclusion proofs can be checked without privileged access. To support supply-chain review, the bundle also contains an SBOM and signature material for the PAB and sidecar. Finally, it embeds the replay kit for this release, allowing third parties to reproduce monitor decisions and to diagnose any drift across versions using the exact parameters recorded in production.

6.2 Per-emission certificate (runtime)

```
{
  "schema": "cert-v1",
  "bundle_id": "pab-2025.09.01+build17",
  "sidecar_build": "1.3.0",
  "epoch": 42,
  "event": "net.send",
  "args_fingerprint": "sha256:...",
  "policy_hash": "sha256:...",
  "proof_hash": "sha256:...",
  "automata_hash": "sha256:...",
  "labeler_hash": "sha256:...",
  "ni_monitor": "accept|reject|inapplicable",
  "permit_decision": "accept|reject",
  "path_witness_ok": true,
  "label_derivation_ok": true,
  "attestation": null,
  "transparency_ref": "rekor:logindex/123456",
  "sig": "dsse:..."
}
```

6.3 SLOs

Operational assurances are only meaningful when they are measured continuously. For each service, teams publish service-level objectives that cover certificate verification success rate, replay determinism under the declared egress profile, monitor overhead at the 95th percentile, and adherence to the reject budget during enforce mode. Thresholds for alerting are defined alongside these targets so that regressions trigger paging rather than remaining latent in quarterly reviews. The evidence bundle records the SLO definitions and the measurement queries used to compute them, which ensures that auditors and operators evaluate the system against the same criteria.

SLO	Target
Certificate verification rate	99.99% per release window
Replay determinism (declared profile)	99.9%
Monitor overhead	p95 < 2 ms / event
Reject budget adherence	$\geq 99\%$ of periods
Inclusion-proof latency (transparency)	p95 < 200 ms

6.4 Dashboards

Dashboards provide a live view of assurance posture and highlight where policy or implementation needs attention. Core panels show acceptance and denial rates by clause, distributions of monitor latency, quota consumption over time, and the incidence of proposed-versus-live diffs during shadow mode. Additional views track certificate verification outcomes, hash drift between deployed artifacts and release records, break-glass activations with reasons and TTLs, and the health of deterministic egress (including any deviation from the declared chunk and flush parameters). When residual-channel measurements are part of the program, the dashboard presents the current capacity estimates and the configured padding profile so that stakeholders can see how mitigation settings affect both risk and availability.

Panel	What it shows / why it matters
Monitor verdicts	Accept/deny rates by clause, service, principal (detect drift)
Reject budget	Denies vs. budget; budget resets; break-glass invocations
Latency	p50/p95/p99 monitor overhead; tail amplification under load
Replay determinism	Pass/fail by profile; drift events; seed/locale mismatches
Evidence health	Cert verification success; transparency inclusion latency
Labeler integrity	Witness mismatches, ‘unknown fields’ rejections

7 Runbooks

7.1 CI gates

Continuous integration blocks any release that cannot be tied back to its proofs and that cannot be reproduced by an independent party. The pipeline first verifies artifact integrity by checking that the `policy_hash`, `proof_hash`, and `automata_hash` embedded in the PAB match the commit identifiers of the source specification, the mechanized proof objects, and the generated monitor tables built in the same run. A mismatch indicates drift between what was proved and what will run and is treated as a hard failure. The pipeline then executes the conformance suite end to end. This suite replays canonical “good” and “bad” traces against the generated monitor and compares outcomes to pass/fail oracles; when a test fails, a counterexample shrinker reduces the trace to a minimal, actionable prefix so engineers can diagnose the issue quickly. Performance is guarded by a regression check that compares the current build’s monitor overhead to a pinned baseline; the build must demonstrate that the *p95* event-level latency has not regressed by more than 10%. If intentional changes affect performance, the baseline is updated only after review and a successful shadow run. Finally, the pipeline enforces evidence completeness: the release must include a software bill of materials (SBOM), signed envelopes for the PAB and sidecar, a replay kit sufficient to re-derive monitor verdicts, and representative certificate samples (with transparency-log references where applicable). A release without these materials cannot be audited and is therefore rejected.

7.2 Rollout: clause by clause

Operational rollouts proceed incrementally so that availability is preserved while assurance improves. Teams begin with low-risk, high-value clauses that fail closed with minimal user

impact, for example “deny raw-token egress,” explicit declassification prerequisites, and adapter interlocks. Higher-churn clauses such as quotas and fine-grained rate shaping follow only after the low-risk set has stabilized. Each clause advances through the modes **observe** → **shadow** → **enforce**. During **observe**, teams confirm that the effect surface is fully mediated and that evidence is produced for every emission. During **shadow**, the system logs both live and proposed monitor decisions and engineers tune thresholds until false-denies fall below target. Promotion to **enforce** requires at least seven consecutive days without regression in shadow, with acceptance/denial diffs, monitor errors, and replay drift all within agreed bounds. Rollouts are staged per service or tenant, and each enforced clause is paired with a documented reject budget and on-call coverage so that transient spikes do not degrade availability unexpectedly.

7.3 Break-glass

Emergency overrides are tightly governed and short-lived. Any break-glass action requires M-of-N maintainer signatures and carries a time-to-live of at most two hours. Approval immediately pages the on-call team and emits an auditable record containing the reason code, the affected clauses, and the impacted services. While break-glass is active, dashboards annotate all verdicts with the override status, and evidence bundles include the override record so that auditors can reconstruct the exact assurance envelope in effect. Expiry is automatic; early termination is encouraged once a safe configuration is restored. Every activation mandates a postmortem that analyzes why the override was necessary, what compensating controls were in place, and how to prevent recurrence (for example, by refining policy, adjusting reject budgets, or improving adapter resilience).

7.4 Incident response

When a violation or suspected leak is reported, the first action is to preserve the evidence. Operators snapshot certificates, hash-chained logs, and the replay kit for the affected window, and they capture the exact PAB, sidecar build, container images, and configuration pins associated with the run. The team then performs a deterministic replay under the declared egress profile (Section 5) to confirm that outcomes are reproducible and to isolate the minimal counterexample that triggers the monitor or exposes the gap. Remediation targets the narrowest effective layer: a policy refinement, an adapter or schema correction, or a labeler fix with updated witnesses. The proposed patch must pass the conformance suite, clear performance guards, and run in **shadow** without regression before it is promoted to **enforce**. If secrets may have been exposed, rotation is executed in parallel and documented explicitly.

The response concludes with a published postmortem that includes the minimized trace, replay commands, the patch diff, and any changes to SLOs or operational runbooks so that both internal stakeholders and external auditors can verify the resolution independently.

8 Minimal Taint Discipline for RAG

Retrieval-augmented generation (RAG) can be operated safely when every payload that crosses an adapter boundary is tagged and those tags are preserved through the pipeline. At ingress, adapters attach structured metadata that records the *origin* of the content (for example, a repository, bucket, or endpoint), the level of *trust* assigned to that origin (internal, partner, public), the *sensitivity* of the payload (such as public, confidential, or secret), and any *license* constraints that govern downstream use. These tags are embedded both in the per-call event records and in the summaries stored with vector indices so that future queries cannot “forget” where a candidate embedding came from or what obligations it carries. Inside the agent, taint composition is defined by a lattice, and whenever the system joins content from multiple sources it computes the least upper bound of the contributing labels. This rule ensures that one high-sensitivity fragment is sufficient to raise the sensitivity of the whole derived artifact. Emissions to low contexts (for example, network responses or files visible to unprivileged users) are permitted only at explicit declassification points, where policy specifies what transformations or redactions justify release. The SC enforces these obligations online: for every field that appears in an outward-facing event, it verifies witnesses that bind the field to a committed JSON path (via a Merkle proof) and that certify a bounded taint derivation consistent with the declared lattice rules. Any mismatch, missing witness, or unlabeled field causes the emission to fail closed. This discipline yields a practical containment story for RAG without banning embeddings or approximate retrieval, and it makes the release decision auditable long after the fact.

9 Cost Performance

The runtime must be predictable enough for production use while remaining faithful to the formal semantics. Hot paths are therefore specialized: for clauses that dominate traffic, the monitor compiler emits compact code directly from the DFA tables and guards that code with equivalence checks against the canonical interpreter. This approach keeps per-event overhead low without drifting from the reference definition of acceptance. Quotas for calls and bytes are implemented with amortized $O(1)$ sliding-window dequeues; memory usage scales with the configured bound κ and does not grow with the length of the run. We track two

user-facing targets. First, the per-event cost of monitoring, measured at the 95th percentile across representative workloads, should remain below 2 ms. Second, the end-to-end overhead imposed on typical tool paths should stay under 5% relative to an otherwise identical build with mediation disabled. These targets are enforced in continuous integration and revalidated during `shadow` rollouts so that performance regressions become visible before enforcement affects users.

10 Security, Privacy, and Governance Controls

Security begins with complete mediation: no effect is allowed to occur unless it is represented as an event on the alphabet, and no event is considered well-formed unless every field is accompanied by a verifiable witness. This “no effect without an event; no field without a witness” rule prevents silent side channels and ensures that all observable behavior is subject to the same checks. Supply-chain integrity is maintained by signing all release artifacts with DSSE envelopes, publishing a software bill of materials (SBOM) for every build, and recording in-toto provenance for each step that produced the PAB and sidecar binaries. Where appropriate, certificate digests and bundle hashes are included in public transparency logs so that third parties can verify inclusion independently. Privacy requirements are respected without sacrificing reproducibility: logs that could reveal sensitive payloads are redacted at record time, while fixture bundles preserve enough structure to replay decisions deterministically. Finally, governance follows an open, versioned process. Clause semantics are proposed and stabilized through numbered RFCs that include reference monitors and conformance tests; deprecations are announced with migration guides and timelines so operators can adopt improvements without service disruption.

11 Limitations and Risks

The guarantees described here depend on explicit operational assumptions. Composing local monitors safely requires that concurrent event streams satisfy an independence discipline and that services introduce serialization points where necessary; when those conditions are violated, enforcement may diverge from the global intent. The scope of Provability Fabric is deliberately bounded: it constrains interfaces and observable effects but does not claim to prove global semantic properties of complex model internals. Covert channels are *bounded* by deterministic egress profiles and optional padding, not eliminated outright; residual capacity persists and must be measured and monitored over time. The taint discipline, while practical, requires disciplined tagging at adapter boundaries and diversity in labeler implementations so

that a single bug cannot undermine the guarantee. Legacy tools that were not designed with labeling in mind may require shims or conservative wrappers, which can increase operational overhead during the transition period.

12 Roadmap

Near-term work focuses on making the system easier to deploy at scale and easier to audit across organizations. A fleet-wide rollout cookbook will provide prescriptive recipes for Kubernetes, serverless, and edge environments, including default network and filesystem mediation hooks, recommended reject budgets, and health probes. To illuminate the trade-offs in deterministic egress, we will publish harnesses that calibrate residual timing and length channels and release public capacity curves that relate padding strategies to false-block rates. First-party dashboards will graduate from reference code to supported products, with SLO automation that opens incidents when certificate verification or replay determinism dips below targets and with auto-regression gates that halt promotions when performance drifts. For edge deployments and regulated settings, a TEE-backed SC will add remote attestation, sealed evidence, and verification SLOs for quote validation so that operators can reason about *how* the code ran in addition to *what* it emitted.

13 Conclusion

Operating the Provability Fabric is ultimately a practice rather than a one-time integration. The value comes from turning proofs into an everyday habit: enforce policies online at the exact points where effects occur, measure outcomes continuously against explicit SLOs, replay executions deterministically when something looks wrong, and publish evidence that others can verify without trusting your internals. By rolling out clause by clause and by treating conformance and replay as part of day-two operations, organizations can raise their assurance floor without sacrificing availability. In that steady state, PF is not an aspiration or a slogan; it is the routine safety layer that keeps agent behavior boringly reliable.

Appendix A: Glossary

The glossary below fixes the terminology used throughout the white paper so that implementers, auditors, and operators share the same referents. Each entry uses operational language and points to the evidence or artifacts that make the concept verifiable in practice.

PAB

The *Provable Agent Bundle* is a signed, versioned package that travels with the agent from build to production. It contains the agent binary and configuration, the compiled monitor tables, the human-readable policy plus its compiled core, the machine-checkable proof artifacts, and the evidence schema that defines how certificates and logs are produced. A PAB allows any third party to reconstruct which proofs and monitors were supposed to govern a run and to verify that the deployed bits match those proofs.

Sidecar

The *sidecar* is the mediator process that intercepts all effectful operations and issues admit or deny verdicts according to the compiled monitor. It instruments filesystem, network, process, and tool boundaries (FS/NET/PROC/TOOL), enforces the deterministic egress profile, and emits tamper-evident logs and certificates. Because every external effect must pass through the sidecar, assurance statements about runtime behavior remain meaningful and auditable.

Alphabet (Σ)

The *alphabet* is the typed set of event labels and arguments that represents every observable effect at the mediation boundary. By requiring that “no effect occurs without a corresponding event on Σ ,” we ensure that specifications, monitors, and replays all speak about the same objects. This alignment is what turns proofs into operational guarantees.

Reject budget

A *reject budget* is a service-level guardrail that caps the number of denials permitted in a window while enforcement is being rolled out. It preserves availability during policy hardening by ensuring that accidental over-blocking triggers alarms and break-glass rather than prolonged outages. Budgets are published, monitored, and tied to on-call procedures.

Replay kit

A *replay kit* is the bundle that rehydrates the environment required to re-derive verdicts for a recorded run. It includes fixed egress parameters (chunk size, flush cadence, locale, timezone), pinned fixtures, canonical traces, and pass/fail oracles. Auditors use the kit to confirm that the same monitor, given the same inputs, makes the same decisions byte-for-byte.

Appendix B: Example Policy & Clause Diff

The following sketch illustrates how a confidentiality policy is expressed in the DSL and how deterministic egress parameters are declared alongside authorization and declassification rules. The example authorizes a small set of tools, explicitly denies a sensitive operation, places a bounded quota on email drafting, and fixes the egress profile so replays are faithful.

```
policy Confidentiality {
  allow tool.call[name in {calendar.read, email.draft}];
  deny tool.call[name == payments.transfer];
  quota email.draft per 5m <= 10;
  declass redact(token) where token in secret;
  egress profile P1 { chunk=4096, flush_ms=40, locale="C.UTF-8", tz="UTC" }
}
```

The transition from **shadow** to **enforce** is recorded as a small, auditable change to the clause metadata rather than as a wholesale policy rewrite. In practice, this change also references the applicable reject budget so that SREs can reason about blast radius during the cutover.

```
clause: "quota email.draft per 5m <= 10" mode: shadow

clause: "quota email.draft per 5m <= 10" mode: enforce reject_budget: 25/h
```

Appendix C: Replay Bundle Schema

A replay bundle declares the deterministic egress profile, pins the environment, and lists the traces and oracles required to re-derive monitor verdicts. Hashes bind the bundle back to the governing policy, proof, and automata so that a verifier can confirm they are checking the correct artifacts.

```
{
  "profile": "EGRESS-DET-P1",
  "env": {"locale": "en_US", "timezone": "UTC", "chunk_bytes": 4096, "flush_ms": 40},
  "fixtures": {"corpus": "s3://.../demo-corpus.tar.zst"},
  "traces": [
    {"id": "trace-001", "events": "...", "cert_refs": ["..."]},
    {"id": "trace-002", "events": "...", "cert_refs": ["..."]}
  ],
  "oracles": {"pass": ["trace-001"], "fail": ["trace-002"]},
  "hashes": {"policy_hash": "sha256:...", "proof_hash": "sha256:...", "automata_hash": "sha256:..."}
```

```
}
```

In this schema, **profile** names the egress profile that fixes chunking, flushing, and locale/timezone; **env** captures the exact runtime pins needed for reproducibility; **fixtures** points to the inputs used during the run; **traces** enumerates the events and the certificates observed; **oracles** declare which traces must pass and which must fail for conformance; and **hashes** tie the bundle to the release’s policy and proof artifacts.

Appendix D: CI Gate Snippets

The commands below illustrate how a continuous-integration pipeline enforces traceability, correctness, and performance before a release is promoted. Each step fails the build on drift or regression, turning assurance from a best-effort practice into a gate.

```
Fail build if monitor/proof hashes drift
```

```
verify_hashes --policy policy.yaml --proof proofs.olean --dfa monitor.json
```

```
Run conformance oracle
```

```
conform --bundle pab.tgz --suite tests/ --on-fail shrink --seed 1337
```

```
Performance budget
```

```
bench-monitor --p95-threshold-ms 2.0 --regression 10%
```

The **verify hashes** step ensures that the monitor and proof artifacts embedded in the PAB match the sources under version control. The **conform** step runs good and bad traces against the reference oracles and minimizes any failing counterexample for triage. The **bench-monitor** step enforces a performance SLO by rejecting builds whose p95 latency exceeds the threshold or whose regression exceeds the allowed percentage.

Appendix E: Dashboard Panels (PromQL sketches)

Operators need live visibility into whether enforcement remains within guardrails, whether monitors are introducing unacceptable latency, and whether replays remain faithful. The PromQL sketches below show how to compute these signals from exported metrics.

Denies vs budget

```
sum by(service)(rate(sidecar_denies_total[5m])) /  
sum by(service)(sidecar_reject_budget_per_minute)
```

Monitor overhead

```
histogram_quantile(0.95, sum by(le)(rate(sidecar_monitor_latency_ms_bucket[5m])))
```

Replay determinism

```
sum(rate(replay_drift_events_total[1h])) / sum(rate(replayed_traces_total[1h]))
```

The first panel tracks how quickly a service is spending its reject budget so that on-call engineers can react before user-visible impact occurs. The second computes the 95th percentile of monitor latency to catch regressions early. The third reports the fraction of drift events observed during automated replays, which should remain near zero when the egress profile and fixtures are correctly pinned.

Appendix F: Build/Runtime Figure

The figure below summarizes how specifications, proofs, and monitors are produced at build time and how the sidecar mediates effects and emits evidence at runtime. Arrows indicate the provenance of artifacts and the flow of enforcement and logging during execution.

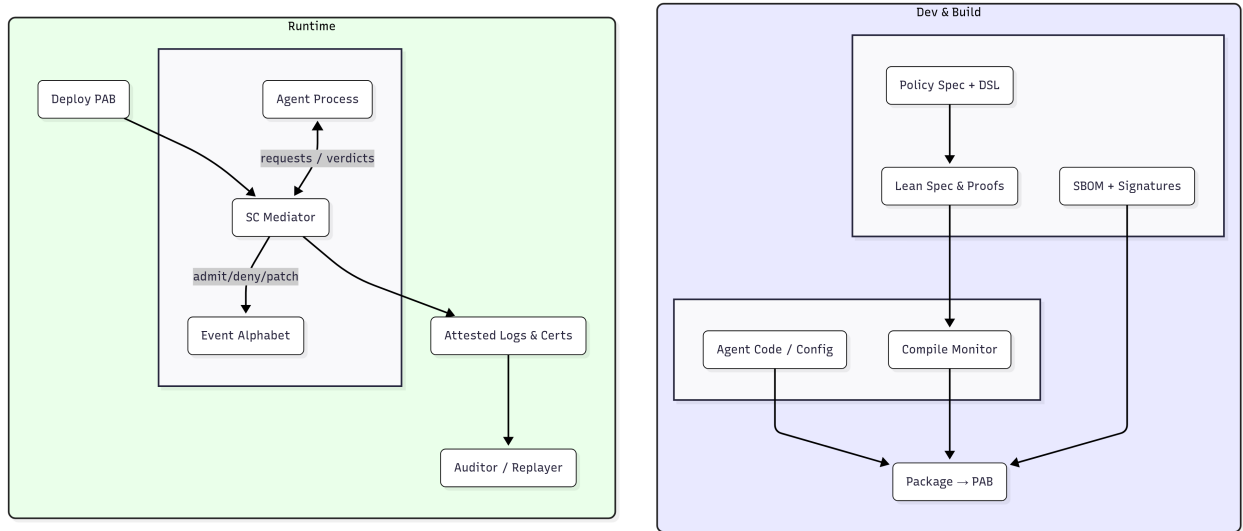


Figure 1: Build-time artifacts and runtime mediation in PF.